(첨부 5)

과제번호	201○-○○

2026년도 전자공학과
종합설계 프로젝트 최종 보고서

YOLOv5 기반 객체 추종 및 장애물 회피 운반 로봇

YOLOv5-Based Object Tracking and Obstacle Avoidance Transport Robot



청주대학교 이공대학 전자공학과				
제출일자	2026년 5월 11일			
팀 구성원	학번	2021012296	성명	정태웅
	학번	2021012255	성명	유제민
지도교수	김동현			

목 차

요약 ····· 3

I. 서론 ····· 4
I-1. 목표 ·····
I-2. 추진 배경 및 필요성 ·····
I-3. 연구/개발 내용 ·····
I-4. 기대효과 및 활용방안 ·····

Ⅱ. 설계 ····· 4
Ⅱ-1. 관련이론 및 개발환경 ·····
Ⅱ-2. 개념설계 ·····
Ⅱ-3. 상세설계 ·····

Ⅲ. 개발 일정 및 자원 관리 ····· 5
Ⅲ-1. 프로젝트 팀 구성 및 역할 ·····
Ⅲ-2. 개발 일정 ·····
Ⅲ-3. 프로젝트 비용 ·····

Ⅳ. 제작 ····· 6
Ⅳ-1. 구현 ·····
Ⅳ-2. 성능평가 ·····
Ⅳ-3. 결과 및 성능 ·····

Ⅴ. 결론 ····· 6

참고문헌 ····· 8

부록 ····· 9

과제 요약서

과제번호	201○-○○

작품 제목	YOLOv5 기반 객체 추종 및 장애물 회피 운반 로봇
	YOLOv5-Based Object Tracking and Obstacle Avoidance Transport Robot

제원		
하드웨어 (H/W)	크기	26cm * 36cm * 34cm
	무게	8.47kg
	프로세서	ATmega 128, NVIDIA Jetson Nano
스프트웨어 (S/W)	OS	Windows, Ubuntu 18.04
	개발TOOL	ATmel studio, Android Studio

개발 기간
2026.01.08 ~ 2026.05.07

개발 소요비	
재료비	592810
제작비	0
기타 경비	0
총 소요비	592810

외관



과제 수행자	성명	정태웅			
	Phone	0108646			
	Email	wjdxodnd aver.com	er.com		
	담당	sw/hw 개발	sw/hw 개발		

이와 같이 2026년도 전자공학과 종합설계 프로젝트 최종 보고서를 제출합니다.

2026**년 5월 11일**

과제 수행자 : 정태웅 **(인)**

과제 수행자 : 유제민 (인)

지도교수 : 김동현 (인)

청주대학교 공과대학 전자공학과

I. 서론

장애인, 노약자, 임산부 등 이동이 불편한 사용자는 짐 운반 과정에서 불편함을 겪을 수 있으며, 최근 물류·서비스 로봇 분야에서도 사용자 추종형 운반 시스템의 활용이 증가하고 있다. 이에 본 작품은 YOLOv5 기반 객체 추종 기술과 장애물 회피 기능을 적용하여 사용자 편의성과 이동 보조 기능을 향상시키는 스마트 운반 로봇 시스템을 구현하고자 한다.

1. 목표

본 프로젝트의 목표는 카메라 기반 객체 인식 및 추적 기술을 이용하여 사용자를 실시간으로 따라가며 물품을 운반할 수 있는 스마트 운반 로봇 시스템을 구현하는 것이다. 구체적으로는 첫째, YOLOv5를 이용하여 사람 객체를 실시간으로 검출하고, 둘째, DeepSORT Re-ID 알고리즘을 적용하여 다중 객체 환경에서도 특정 사용자를 지속적으로 식별·추적하며, 셋째, Jetson Nano와 ATmega128 간 UART 통신을 통해 주행 방향과 속도를 제어하는 것이다. 또한 전방, 좌전방, 우전방의 초음파 센서를 이용하여 근거리 장애물을 감지하고, 장애물 발생 시 우선 회피 기동을 수행하도록 하여 보다 안정적인 실내 주행 성능을 확보하고자 하였다. 최종적으로는 사용자가 별도의 복잡한 조작 없이 짐을 적재한 로봇을 동행형으로 활용할 수 있는 시스템 구현을 목표로 한다.

2. 추진 배경 및 필요성

본 설계 프로젝트의 제목은 YOLOv5 기반 객체 추종 및 장애물 회피 운반 로봇이다. 본 프로젝트는 카메라 기반 객체 인식 기술과 초음파 센서를 활용하여 사용자를 실시간으로 인식하고 추종하며, 주행 중 장애물을 감지하면 이를 회피할 수 있는 지능형 운반 로봇 시스템을 구현하는 것을 내용으로 한다. 또한 Bluetooth 기반 컨트롤러 앱을 통해 수동 모드와 객체 추종 모드를 전환할 수 있도록 하여 사용자 편의성을 높이고자 하였다.

기존의 운반 로봇은 사용자가 직접 조작하거나 사전에 지정된 경로를 따라 이동하는 방식이 대부분이어서, 사용자의 위치 변화에 능동적으로 대응하기 어렵다는

한계가 있다. 특히 장애인, 노약자, 임산부와 같이 이동이 불편한 사용자는 짐을 운반하는 과정에서 불편을 겪을 수 있으며, 이를 보조할 수 있는 사용자 추종형 운반 시스템의 필요성이 증가하고 있다. 또한 최근 물류·서비스 로봇 분야에서는 단순 반복 작업을 넘어 사용자 중심의 이동 보조와 편의 기능이 중요해지고 있다.

이에 본 프로젝트에서는 YOLOv5와 DeepSORT 기반 객체 인식 및 추적 기술을 이용하여 특정 사용자를 지속적으로 추종하고, 초음파 센서를 통해 전방, 좌전방, 우전방 장애물을 감지하여 회피 동작을 수행하는 운반 로봇 시스템을 설계하고자 하였다. 본 시스템은 사용자가 별도의 복잡한 조작 없이 짐을 운반할 수 있도록 도와줄 뿐만 아니라, 실내 환경에서의 이동 편의성과 주행 안정성을 함께 향상시킬 수 있다는 점에서 필요성이 있다. 또한 Jetson Nano와 ATmega128을 이용한 상위·하위 제어 구조를 통해 임베디드 시스템, 객체 인식, 센서 응용, 통신 제어 기술을 종합적으로 적용할 수 있다는 점에서도 교육적·기술적 가치가 크다.

(1) 프로그램 구현에 사용한 시스템 환경

본 프로젝트는 Jetson Nano를 상위 제어부로, ATmega128을 하위 제어부로 사용하여 구성하였다. Jetson Nano에서는 Ubuntu 18.04 환경에서 YOLOv5와 DeepSORT를 이용한 객체 검출 및 추적을 수행하고, ATmega128에서는 UART 통신을 통해 수신한 제어 데이터를 기반으로 모터와 센서를 제어하였다. 또한 Bluetooth 모듈과 안드로이드 컨트롤러 앱을 연동하여 수동 모드와 객체 추종 모드 전환 기능을 구현하였다.

가) 시스템 파라미터:

시스템 변수	값	시스템 변수	값
상위 제어부	NVIDIA Jetson Nano	하위 제어부	ATmega128
운영체제	Ubuntu 18.04	카메라	CSI 카메라
객체 검출 알고리즘	YOLOv5	객체 추적 알고리즘	DeepSORT
통신 방식	UART, Bluetooth	모터 제어 방식	PWM
센서 구성	초음파 센서 3개	개발 도구	Atmel Studio, Android Studio
앱 제어 방식	Android Bluetooth 앱	주행 모드	수동 제어, 객체 추종

3. 연구 / 개발 내용

본 프로젝트는 크게 수동 제어 시스템, 객체 추종 시스템, 장애물 회피 시스템으로 구

성되며, 단계별 수행 내용은 다음과 같다.

3.1 시스템 설계 및 수동 제어 구현

Jetson Nano, ATmega128, Bluetooth 모듈, 카메라, 초음파 센서, 모터 드라이버를 연동한 전체 시스템을 설계하였다. 또한 안드로이드 앱과 Bluetooth 모듈을 연동하여 수동 모드와 객체 추종 모드를 선택할 수 있도록 하였으며, 수동 모드에서는 전진, 후진, 좌회전, 우회전, 정지 기능을 구현하였다.

3.2 YOLOv5 및 DeepSORT 기반 객체 추종 시스템 구현

Jetson Nano와 카메라를 이용하여 사람 객체를 실시간으로 검출하고, DeepSORT Re-ID 알고리즘을 통해 특정 타겟을 지속적으로 추적하도록 구현하였다. 또한 바운딩 박스의 위치와 크기 정보를 기반으로 사용자의 상대적 위치와 거리를 판단하여 주행 방향과 속도를 결정하였다.

3.3 모터 제어 및 장애물 회피 시스템 구현

Jetson Nano에서 생성된 제어 데이터를 UART 패킷 형태로 ATmega128에 전달하고, ATmega128은 이를 해석하여 좌우 모터를 PWM 방식으로 제어하도록 구현하였다. 또한 전방, 좌전방, 우전방 초음파 센서를 통해 장애물을 감지하고, 장애물 존재 시 객체 추종보다 회피 동작을 우선 수행하도록 설계하였다. 추가적으로 앱의 타겟 재지정 기능을 통해 새로운 객체를 다시 선택할 수 있도록 구현하였다.

4. 기대효과 및 활용방안

본 프로젝트를 통해 사용자가 직접 조작하지 않아도 로봇이 사용자를 따라 이동하며 물품을 운반할 수 있으므로 작업 편의성과 이동 효율을 높일 수 있다. 특히 장애인, 노약자, 임산부와 같이 이동이 불편한 사용자의 짐 운반 보조 수단으로 활용할 수 있어 생활 편의성 향상에 기여할 수 있다. 또한 초음파 센서를 이용한 장애물 감지 및 회피 기능을 추가함으로써 실내 환경에서의 주행 안정성을 높이고 충돌 위험을 줄일 수 있다.

기존의 고정 경로 기반 운반 로봇은 정해진 경로에서만 동작하는 한계가 있었으나, 본 프로젝트는 YOLOv5와 DeepSORT 기반 객체 추종 기능을 적용하여 사용자를 직접 인식하고 추적할 수 있도록 구현하였다. 이를 통해 사용자 이동에 따라 유연하게 주행할 수 있으며, 사용자 중심 서비스 로봇으로의 활용 가능성을 높였다.

본 시스템은 병원, 학교, 공공시설, 공장, 창고 등에서 소형 물품 운반 로봇으로 활용 가능하며, 향후 스마트 물류 로봇, 보행 보조 로봇, 사용자 추종형 서비스 로봇 등으로 확장 개발할 수 있다.

Ⅱ. 설계

1. 관련이론 및 개발환경

관련 이론

1) Object Detection & Following(객체 인식 & 추종)

-YOLO

-DeepLearning

-DeepSORT

개발 환경

- Ubuntu 18.04
- Android Studio
- Atmel Studio

2. 개념설계

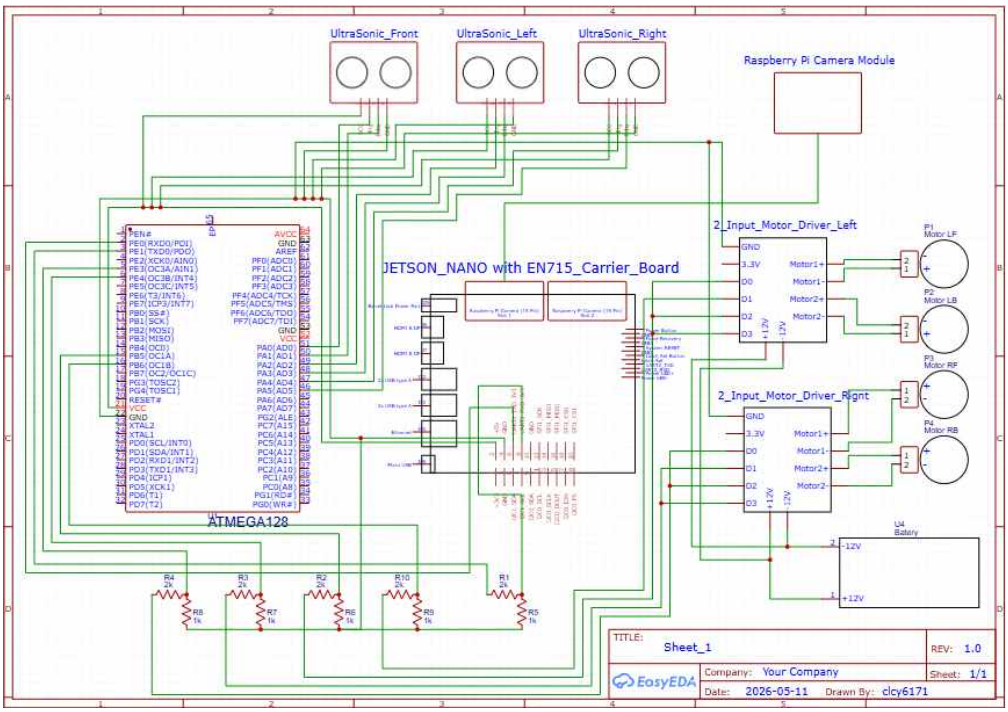
본 프로젝트는 사용자를 자동으로 추종하며 장애물을 회피할 수 있는 운반 로봇 시스템을 목표로 설계하였다. 시스템은 상위 제어부와 하위 제어부를 분리한 구조로 구성하였다.

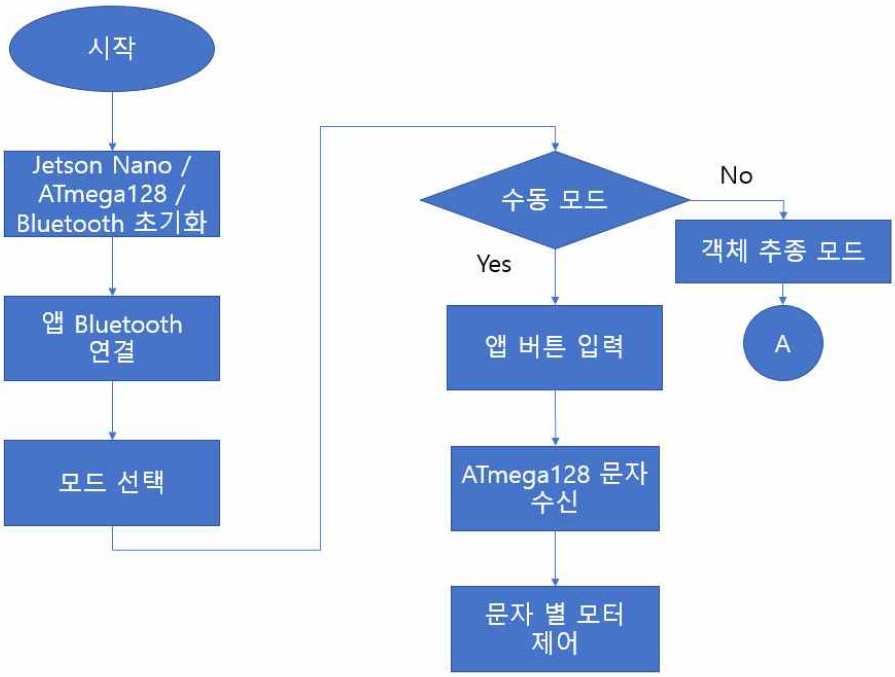
상위 제어부인 Jetson Nano는 카메라 영상을 입력받아 YOLOv5를 통해 사람 객체를 검출하고, DeepSORT 알고리즘을 이용하여 특정 타겟을 지속적으로 추적한다. 이후 타겟의 위치와 크기 정보를 기반으로 전진, 정지, 후진 및 좌우 방향 보정에 필요한 제어값을 계산한다.

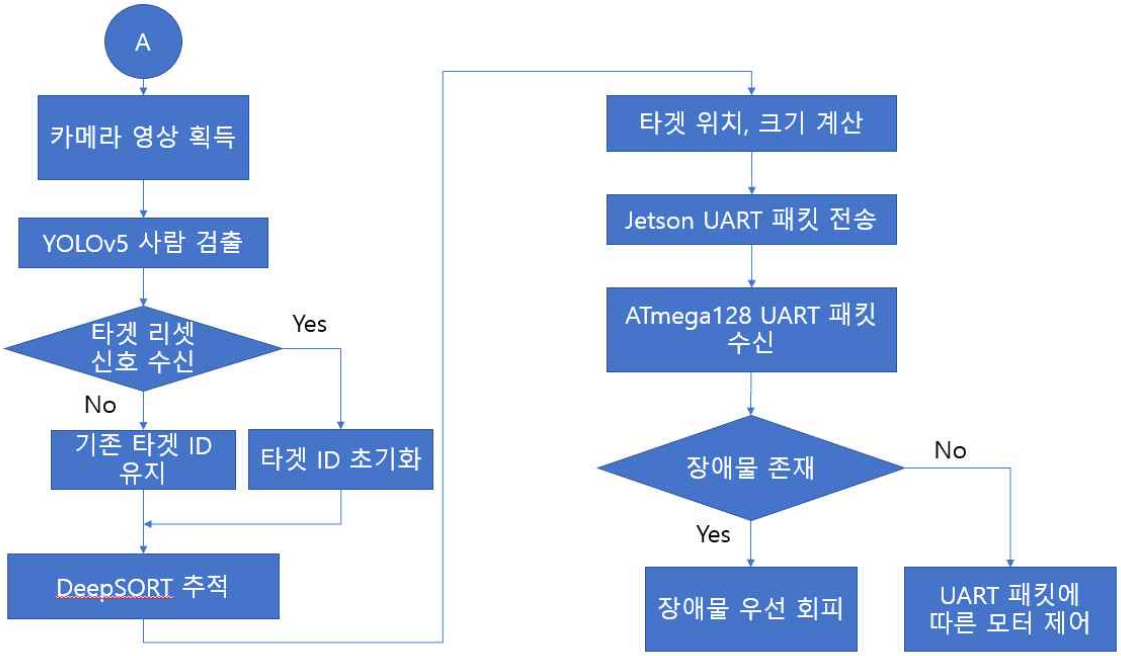
하위 제어부인 ATmega128은 Jetson Nano로부터 UART 패킷 형태의 제어 데이터를 수신하고, 좌우 모터를 PWM 방식으로 제어하여 로봇을 구동한다. 또한 Bluetooth 기반 안드로이드 앱과 연동하여 수동 제어 모드와 객체 추종 모드를 전환할 수 있도록 설계하였다.

장애물 회피 기능은 ATmega128에서 수행하며, 전방 및 좌, 우전방 초음파 센서를 통해 장애물 존재 여부를 판단한다. 장애물이 감지될 경우 객체 추종 명령보다 회피 동작을 우선 수행하도록 설계하여 주행 안정성을 향상시켰다.

3. 상세설계


UltraSonic_Front
UltraSonic_Left
UltraSonic_Right
Raspberry Pi Camera Module
JETSON_NANO with EN715_Carrier_Board
2_Input_Motor_Driver_Left
2_Input_Motor_Driver_Right
ATMEGA128
Motor LF
Motor LB
Motor RF
Motor RB
Battery
TITLE: Sheet_1
REV: 1.0
Company: Your Company
Sheet: 1/1
Date: 2026-05-11
Drawn By: clcy6171
EasyEDA


시작
Jetson Nano / ATmega128 / Bluetooth 초기화
앱 Bluetooth 연결
모드 선택
수동 모드
No
Yes
객체 추종 모드
A
앱 버튼 입력
ATmega128 문자 수신
문자 별 모터 제어



Ⅲ. 개발 일정 및 자원 관리

1. 프로젝트 팀 구성 및 역할

이름	역할
정태웅	- 객체 인식 및 추적 시스템 구현 - Jetson Nano 환경 구축 및 시스템 통합
유제민	- 임베디드 제어 및 모터 구동 구현 - Bluetooth 및 Android 앱 연동

2. 개발 일정

항목	세부내용	1 월	2 월	3 월	4 월	5 월	비고
아이디어도출	아이디어 선정	■					
	구현 가능성 평가	■					
관련분야연구	주요 기술 연구		■	■			
	관련 시스템 분석		■	■			
설계	시스템 설계				■		
구현	코딩 및 모듈 테스트				■		
테스트	시스템 테스트					■	

3. 프로젝트 비용

순번	물품 명	수량	단가	총액
1	Jetson Nano with EN715	1	412500	412500
2	HC-SR04	3	1210	3630
3	감속기어모터 RB-35GM 05TYPE (12V)	4	24970	99880
4	70W+70W 5A L298N 듀얼채널 DC모터 드라이버 [ZK-5AD]	2	6380	12760
5	라즈베리파이 카메라모듈 V2	1	26400	26400
6	[EL-RTB] 65mm 로봇휠 Set(2EA) [크롬블루]	2	13,820	27,640
7	다이소 캠핑용 수납 박스	2	5000	10000
총 합 계		15	490280	592810

Ⅳ. 제작

1. 구현

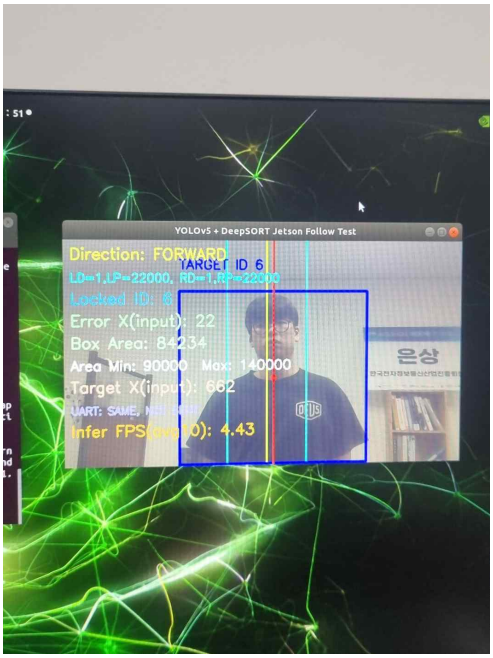
Object Detection & Following(객체 인식 & 추종)

Jetson Nano 환경에서 YOLOv5 및 DeepSORT가 정상적으로 동작하도록 개발 환경을 구축하였으며, CSI 카메라 영상을 이용하여 사람 객체를 실시간으로 검출하고 추적할 수 있도록 구현하였다. YOLOv5를 통해 사람 객체를 검출한 뒤 DeepSORT를 이용하여 동일 타겟의 ID를 유지하며 지속적으로 추적하도록 하였고, 타겟 리셋 신호가 입력되면 기존 locked_target_id를 초기화하여 새로운 대상을 다시 지정할 수 있도록 하였다.

또한 추적된 타겟의 중심 좌표 Target X(input)와 바운딩 박스 면적 Box Area를 이용하여 추종 동작을 판단하도록 구현하였다. Box Area가 STOP_AREA_MIN보다 작을 경우 전진, STOP_AREA_MIN 이상 STOP_AREA_MAX 이하일 경우 정지, STOP_AREA_MAX를 초과할 경우 후진하도록 설정하였다. 또한 Target X(input)가 화면 중앙 기준 좌측 또는 우측 데드존을 벗어날 경우 방향 보정을 수행하여 FORWARD-LEFT, FORWARD-RIGHT, REVERSE-LEFT, REVERSE-RIGHT 동작이 가능하도록 구현하였다.

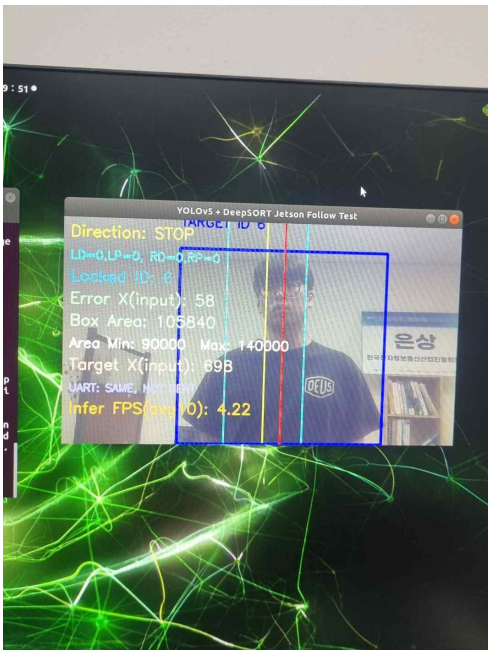
전진

추적된 타겟의 바운딩 박스 면적 Box Area가 STOP_AREA_MIN보다 작을 경우, 사용자가 로봇과 멀리 떨어져 있다고 판단하여 전진(FORWARD) 동작을 수행하도록 구현하였다. 이때 타겟의 중심 좌표 Target X(input)가 화면 중앙 부근의 데드존 범위 내에 위치하면 좌우 보정 없이 직진하도록 하였으며, 데드존을 벗어날 경우 위치 오차에 따라 좌우 모터의 PWM 값을 차등 적용하여 방향을 보정하도록 하였다.



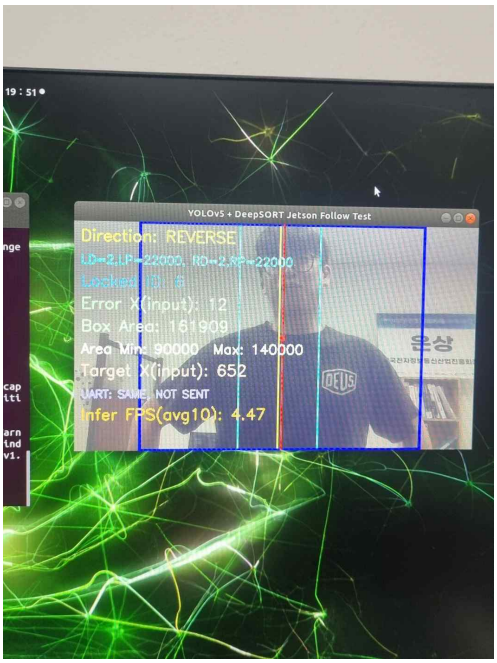
정지

타겟의 바운딩 박스 면적 Box Area가 STOP_AREA_MIN 이상 STOP_AREA_MAX 이하의 범위에 들어올 경우, 사용자가 적정 추종 거리 내에 위치한 것으로 판단하여 정지(STOP) 동작을 수행하도록 구현하였다. 이를 통해 사용자가 너무 멀거나 너무 가까운 상태가 아닌 경우에는 불필요한 전진 및 후진을 방지하고 안정적인 추종 상태를 유지할 수 있도록 하였다.



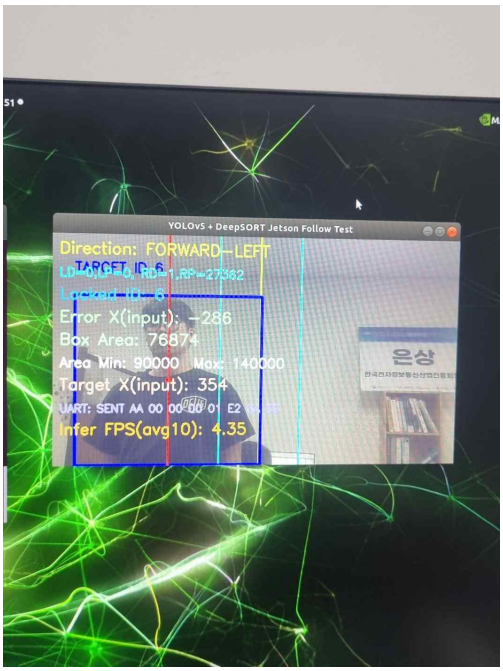
후진

타겟의 바운딩 박스 면적 Box Area가 STOP_AREA_MAX를 초과할 경우, 사용자가 로봇과 지나치게 가까운 상태로 판단하여 후진(REVERSE) 동작을 수행하도록 구현하였다. 이때 타겟이 화면 중앙 부근에 위치하면 양쪽 모터를 동일하게 제어하여 직선 후진을 수행하도록 하였다.



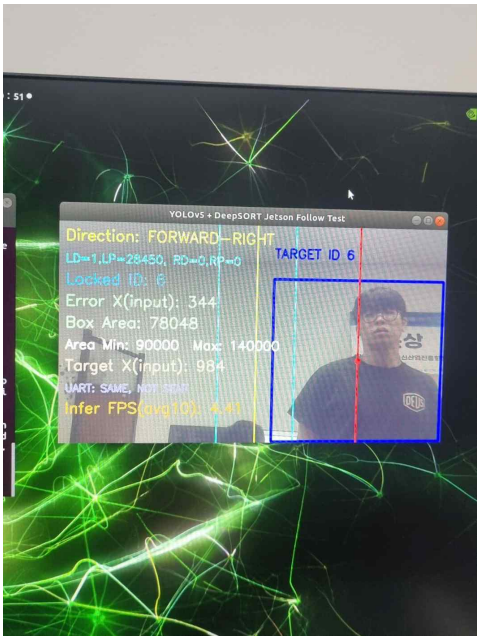
좌회전

타겟의 바운딩 박스 면적이 STOP_AREA_MIN보다 작아 전진이 필요한 상황에서, 타겟 중심 좌표 Target X(input)가 화면 중앙보다 좌측에 위치할 경우 좌측 방향 보정이 필요하다고 판단하여 좌회전(FORWARD-LEFT) 동작을 수행하도록 구현하였다. 이때 전진 방향을 유지하면서 좌측 바퀴를 정지하고 우측 바퀴를 정회전시켜 좌측으로 방향을 보정하도록 하였다.



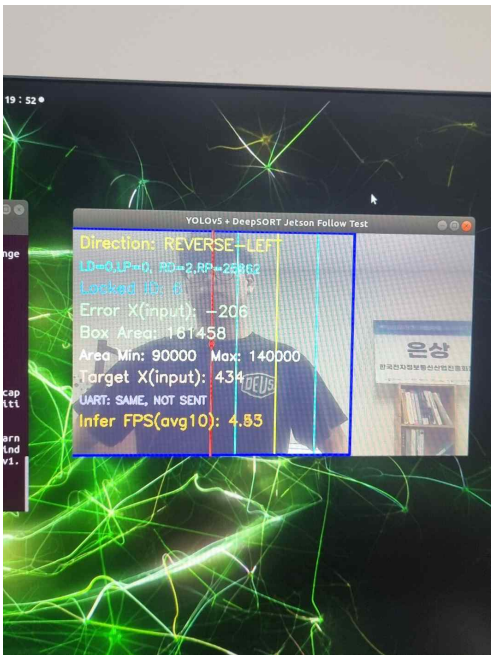
우회전

타겟의 바운딩 박스 면적이 STOP_AREA_MIN보다 작아 전진이 필요한 상황에서, 타겟 중심 좌표 Target X(input)가 화면 중앙보다 우측에 위치할 경우 우측 방향 보정이 필요하다고 판단하여 우회전(FORWARD-RIGHT) 동작을 수행하도록 구현하였다. 이때 전진 방향을 유지하면서 우측 바퀴를 정지하고 좌측 바퀴를 정회전시켜 우측으로 방향을 보정하도록 하였다.



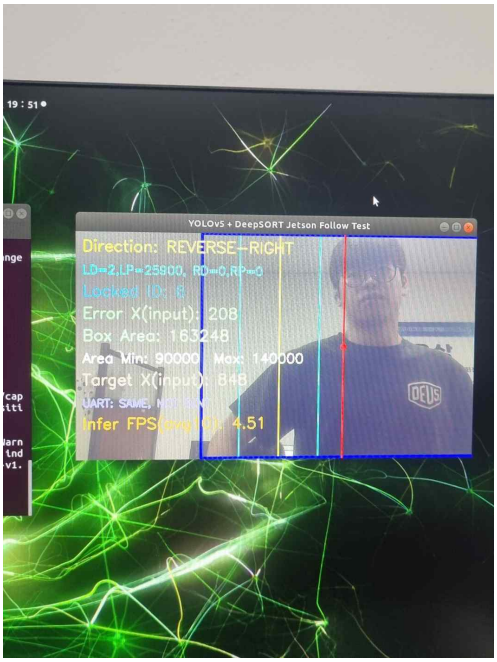
좌후진

타겟의 바운딩 박스 면적이 STOP_AREA_MAX를 초과하여 후진이 필요한 상황에서, 타겟 중심 좌표 Target X(input)가 화면 중앙보다 좌측에 위치할 경우 후진과 동시에 좌측 방향 보정이 필요하다고 판단하여 좌후진(REVERSE-LEFT) 동작을 수행하도록 구현하였다. 이때 좌측 바퀴는 정지하고 우측 바퀴만 역회전시켜 후진하면서 좌측 방향으로 이동하도록 하였다.



우후진

타겟의 바운딩 박스 면적이 STOP_AREA_MAX를 초과하여 후진이 필요한 상황에서, 타겟 중심 좌표 Target X(input)가 화면 중앙보다 우측에 위치할 경우 후진과 동시에 우측 방향 보정이 필요하다고 판단하여 우후진(REVERSE-RIGHT) 동작을 수행하도록 구현하였다. 이때 우측 바퀴는 정지하고 좌측 바퀴만 역회전시켜 후진하면서 우측 방향으로 이동하도록 하였다.



Bluetooth 앱 제어 및 타겟 재지정

Bluetooth 기반 안드로이드 앱을 제작하여 수동 제어 모드와 객체 추종 모드를 선택할 수 있도록 구현하였다. 수동 모드에서는 앱 버튼 입력을 통해 전진, 후진, 좌회전, 우회전 및 정지 기능을 직접 제어할 수 있도록 하였으며, 객체 추종 모드에서는 리셋 버튼 입력 시 타겟 재지정 기능이 동작하도록 구현하였다. 타겟 재지정 시 앱에서 전송된 리셋 문자는 ATmega128을 거쳐 Jetson Nano로 전달되며, Jetson Nano에서는 기존 locked_target_id와 추적 대상 정보를 초기화한 후 새로운 사용자를 다시 선택하도록 구성하였다.



초음파 기반 장애물 회피

장애물 회피 기능 구현을 위해 전방 중앙, 좌전방, 우전방 방향에 초음파 센서를 배치하여 각각 독립적으로 거리 값을 측정하도록 구성하였다. 전방 초음파 센서 값이 기준 거리 이내로 들어오면 장애물이 존재한다고 판단하고, 좌전방 및 우전방 초음파 센서 값을 비교하여 더 넓은 방향으로 회피 동작을 수행하도록 구현하였다. 또한 좌측만 장애물이 감지될 경우 우측 회피, 우측만 장애물이 감지될 경우 좌측 회피를 수행하도록 하였으며, 전방과 좌우 방향 모두 장애물이 감지될 경우에는 후진 동작을 수행하도록 구현하였다. 객체 추종 모드에서는 Jetson Nano에서 전달된 추종 명령보다 초음파 기반 회피 동작을 우선 적용하여 충돌 가능성을 줄이도록 구성하였다.

2. 성능평가

성능평가는 객체 인식 성능, 객체 추종 성능, 수동 제어 성능, 장애물 회피 성능의 네 항목으로 구분하여 수행하였다. 시험은 실제 제작한 로봇, CSI 카메라, Jetson Nano, ATmega128, Bluetooth 앱, 초음파 센서가 모두 연결된 하드웨어 환경에서 진행하였다.

객체 인식 성능은 사용자를 카메라 전방에 위치시킨 후 YOLOv5가 사람 객체를 안정적으로 검출하는지 확인하는 방식으로 평가하였다. 객체 추종 성능은 사용자가 로봇 기준 좌측, 중앙, 우측으로 이동하거나 거리를 변화시켰을 때 로봇이 전진, 정지, 후진 및 좌우 방향 보정을 정상적으로 수행하는지 확인하였다.

수동 제어 성능은 Bluetooth 앱에서 전진, 후진, 좌회전, 우회전, 정지 명령을 입력하고, 각 명령에 따라 좌우 모터가 정상적으로 동작하는지 평가하였다. 장애물 회피 성능은 전방, 좌전방, 우전방에 장애물을 배치한 후 초음파 센서의 거리 측정값을 확인하고, 장애물 감지 시 객체 추종 명령보다 회피 동작이 우선 수행되는지 시험하였다.

시험 결과는 각 항목별 성공 여부, 초음파 센서 거리값, 로봇의 동작 반응을 표로 정리하여 제시한다.

3. 결과 및 성능

- **시험평가 결과**
  - 사용자를 카메라 전방에 위치 후 사람 객체 검출 여부 확인
  - 사용자의 좌·우 이동 및 거리 변화에 따른 추종 동작 확인
  - Bluetooth 앱을 이용한 전진/후진/좌회전/우회전/정지 명령 입력
  - 전방·좌측·우측 장애물 배치 후 초음파 센서 반응 확인
  - 초음파 센서 거리값 실제 측정 결과 확인

〈표Ⅳ-1〉

평가 항목	측정 결과	성공 여부
객체 인식 성능	사람 객체를 안정적으로 검출하였으며 Bounding Box가 지속적으로 유지됨	성공
객체 추종 성능	사용자의 위치 변화에 따라 좌우 회전, 전진 및 정지 동작이 전반적으로 수행되었으나, 일부 프레임 저하로 추종 성능이 완전히 안정적이지는 않았음	보통
수동 제어 성능	입력된 명령에 따라 좌우 모터가 정상적으로 동작함	성공
장애물 회피 성능	장애물 감지 시 추종보다 회피 동작이 우선 수행되었으며 충돌 없이 회피 성공	성공

〈표Ⅳ-2〉

장애물 위치	측정 거리
전방 장애물	10.6cm
좌측 장애물	10.3cm
우측 장애물	10.7cm

- **시험평가 결과에 대한 분석**

객체 인식 시험 결과, 카메라 전방에 위치한 사용자를 전반적으로 안정적으로 검출할 수 있었으며, 조명 변화가 크지 않은 환경에서는 Bounding Box가 비교적 잘 유지되었다. 객체 추종 기능에서는 사용자의 이동 방향에 따라 좌우 방향 보정과 전진, 정지 동작이 수행되었으나, 일부 프레임 저하로 인해 추종 성능이 완전히

일정하게 유지되지는 않았다. 그럼에도 불구하고 전체적으로 사용자를 따라가는 기본적인 추종 동작은 가능함을 확인하였다.

수동 제어 기능은 Bluetooth 통신을 통해 명령 전달이 정상적으로 이루어졌으며, 각 명령에 대응하는 모터 제어가 안정적으로 수행되었다. 이를 통해 비상 상황이나 객체 인식 실패 상황에서도 사용자가 직접 로봇을 제어할 수 있음을 확인하였다.

장애물 회피 기능에서는 초음파 센서가 장애물을 감지하였으며, 장애물 감지 시 객체 추종보다 회피 동작이 우선 실행되었다. 특히 전방 장애물이 존재할 경우 좌우 거리값을 비교하여 회피 방향을 선택하거나, 모든 방향에 장애물이 존재할 경우 후진 동작을 수행함으로써 충돌 가능성을 줄일 수 있었다.

- **설계목표 대비 달성 성과를 기술(개선사항 포함)**

본 프로젝트의 목표는 사용자를 인식하여 자동으로 추종하고, 장애물을 회피하며, 필요 시 Bluetooth 기반 수동 제어가 가능한 지능형 이동 로봇 시스템을 구현하는 것이었다.

성능평가 결과 객체 인식, 수동 제어, 장애물 회피 기능은 전반적으로 정상적으로 동작하였으며, 각 기능이 상호 연동되어 시스템이 동작함을 확인하였다. 객체 추종 기능 또한 사용자를 따라가는 기본 동작은 수행하였으나, 일부 프레임 저하로 인해 추종 성능의 안정성은 다소 미흡한 부분이 있었다. 따라서 본 시스템은 초기 설계 목표를 전반적으로 달성하였으나, 객체 추종 안정성 측면에서는 추가적인 개선이 필요하다고 판단된다.

향후에는 객체 추적 주기 최적화와 연산 경량화를 통해 프레임 저하를 줄이고, 보다 안정적인 추종 성능을 확보할 예정이다. 또한 객체 재식별(Re-ID) 알고리즘과 센서 융합 기술을 보완하여 사용자가 일시적으로 화면에서 벗어나더라도 다시 안정적으로 추적할 수 있도록 개선하고자 한다. 추가적으로 초음파 센서 외에 LiDAR 또는 Depth Camera를 적용하여 거리 측정 정확도와 장애물 인식 범위를 향상시키는 방향으로 발전시킬 수 있다.

Ⅴ. 결론

- **본 과제에서 수행한 내용을 간단하게 요약**

  객체 인식을 통해 직진, 정지, 좌회전, 우회전, 후진 등 사용자의 움직임에 따라 로봇을 구동하고 초음파 센서를 통해 장애물 회피 동작 수행을 한다. 또한 어플리케이션으로 수동 조작을 통해 로봇을 제어할 수 있도록 한다.

- **본 프로젝트에서 달성하지 못한 점이나 문제점 기술**

현재 시스템은 조명 변화가 심하거나 사용자가 카메라 화면에서 빠르게 벗어나는 경우 객체 추종 성능이 저하되는 한계가 존재하였다. 또한 초음파 센서는 측정 각도와 장애물 재질에 따라 거리 오차가 발생할 수 있었으며, 좁은 공간에서는 회피 경로 선택이 제한되는 문제가 있었다.

- **참여한 팀원들의 소감**

  정태웅 - Jetson Nano 환경에서 객체 인식 및 추적 시스템을 구현하면서 비전 기반 제어 시스템의 전체 흐름을 이해할 수 있었다. 환경 구축부터 알고리즘 연동, 시스템 통합까지 직접 수행하며 실질적인 임베디드 AI 개발 경험을 쌓을 수 있었다.

  유제민 - ATmega128 기반 모터 제어와 Bluetooth · Android 앱 연동을 구현하면서 하드웨어와 소프트웨어가 연결되는 과정을 배울 수 있었다. 직접 제어 시스템을 구현하고 디버깅하는 과정에서 임베디드 제어에 대한 이해를 높일 수 있었다.

[참고문헌]

[1] ATmega128A Datasheet: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-8151-8-bit-AVR-Microcontroller-ATmega128A_Datasheet.pdf

[2] AVerMedia EN715 User Manual: https://s3.us-west-2.amazonaws.com/storage.avermedia.com/web_release_www/EN715/User+Manual/AVerMedia_UM_EN715_NO111B_NX211B_TN111B+20240221.pdf

[3] NVIDIA Jetson Nano Developer Kit User Guide: https://developer.nvidia.com/embedded/learn/jetson-nano-2gb-devkit-user-guide

[4] HC-SR04 Ultrasonic Sensor Datasheet: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[5] Ultralytics YOLOv5: https://github.com/ultralytics/yolov5

[6] N. Wojke, A. Bewley, D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” 2017 IEEE International Conference on Image Processing(ICIP), pp. 3645-3649, 2017.

[7] Deep SORT GitHub: https://github.com/nwojke/deep_sort

[8] OpenCV Documentation: https://docs.opencv.org/

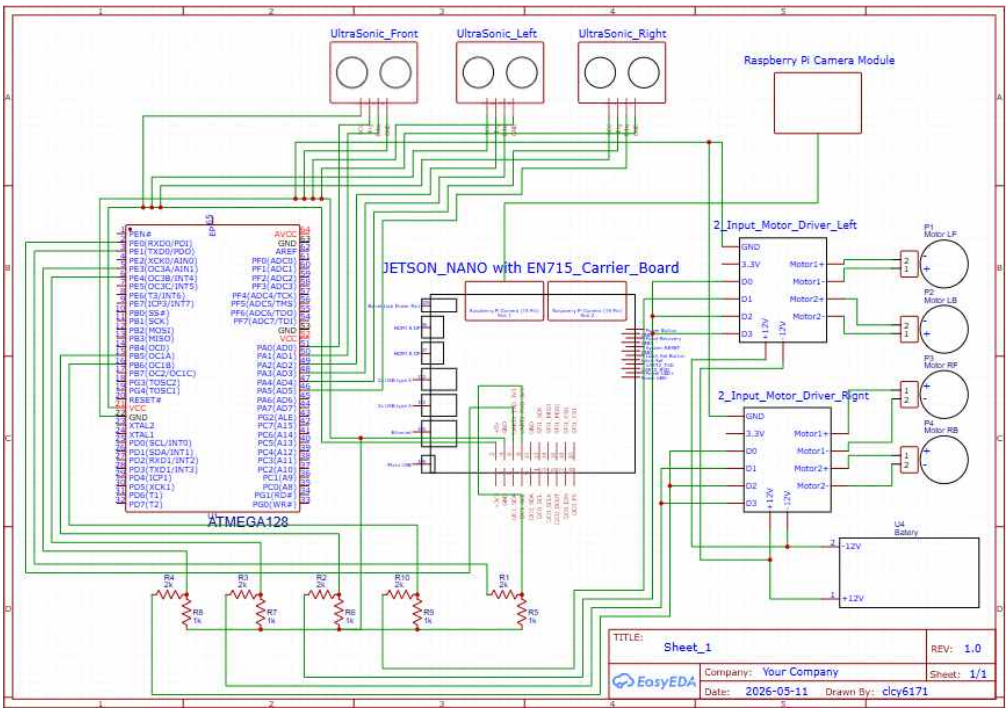
[9] PyTorch Documentation: https://pytorch.org/docs/stable/index.html

프로그램 소스 코드 (또는 전체 회로도)

[사용 가능 실험실]

14-426 신호 및 시스템 실험실

1) 하드웨어 회로도



2-1) 앱 프로그램 manifest 코드

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"
/>

    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />

    <uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
```

```
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Fucking_test2"
        tools:targetApi="31">

        <activity
            android:name="com.example.following_robot.MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>
    </application>

</manifest>
```

2-2) 앱 프로그램 xml 코드

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <ListView
        android:id="@+id/lv_devices"
        android:layout_width="match_parent"
```

```
        android:layout_height="180dp"
        android:dividerHeight="1dp" />

    <Button
        android:id="@+id/btn_scan"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="탐색" />

    <Button
        android:id="@+id/btn_reset"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="타겟 리셋" />

    <Button
        android:id="@+id/btn_mode_toggle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="모드: 수동" />

    <Space
        android:layout_width="match_parent"
        android:layout_height="24dp" />

    <ImageButton
        android:id="@+id/btn_front"
        android:layout_width="90dp"
        android:layout_height="90dp"
        android:layout_gravity="center_horizontal"
        android:background="@android:color/transparent"
        android:contentDescription="전진"
        android:scaleType="fitCenter"
        android:src="@drawable/btn_front" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="horizontal">
```

```
        <ImageButton
            android:id="@+id/btn_left"
            android:layout_width="90dp"
            android:layout_height="90dp"
            android:background="@android:color/transparent"
            android:contentDescription="좌회전"
            android:scaleType="fitCenter"
            android:src="@drawable/btn_left" />

        <ImageButton
            android:id="@+id/btn_stop"
            android:layout_width="90dp"
            android:layout_height="90dp"
            android:layout_marginHorizontal="16dp"
            android:background="@android:color/transparent"
            android:contentDescription="정지"
            android:scaleType="fitCenter"
            android:src="@drawable/btn_stop" />

        <ImageButton
            android:id="@+id/btn_right"
            android:layout_width="90dp"
            android:layout_height="90dp"
            android:background="@android:color/transparent"
            android:contentDescription="우회전"
            android:scaleType="fitCenter"
            android:src="@drawable/btn_right" />
    </LinearLayout>

    <ImageButton
        android:id="@+id/btn_back"
        android:layout_width="90dp"
        android:layout_height="90dp"
        android:layout_gravity="center_horizontal"
        android:background="@android:color/transparent"
        android:contentDescription="후진"
        android:scaleType="fitCenter"
        android:src="@drawable/btn_back" />

</LinearLayout>
```

2-3) 앱 프로그램 main 코드

```
package com.example.following_robot

import android.Manifest
import android.bluetooth.*
import android.content.*
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import android.util.Log
import android.widget.*
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import java.util.UUID

class MainActivity : AppCompatActivity() {

    private val TAG = "BLE"

    private var bluetoothAdapter: BluetoothAdapter? = null
    private val devicesList = ArrayList<BluetoothDevice>()

    private lateinit var listAdapter: ArrayAdapter<String>
    private lateinit var lvDevices: ListView
    private lateinit var btnScan: Button
    private lateinit var btnReset: Button
    private lateinit var btnModeToggle: Button

    private lateinit var btnfront: ImageButton
    private lateinit var btnback: ImageButton
    private lateinit var btnleft: ImageButton
    private lateinit var btnright: ImageButton
    private lateinit var btnstop: ImageButton

    private val SERVICE_UUID =
        UUID.fromString("0000ffe0-0000-1000-8000-00805f9b34fb")
    private val CHARACTERISTIC_UUID =
        UUID.fromString("0000ffe1-0000-1000-8000-00805f9b34fb")

    private var bluetoothGatt: BluetoothGatt? = null
    private var writeCharacteristic: BluetoothGattCharacteristic? = null
```

```
    private var isTrackingMode = false

    private fun hasPermission(permission: String): Boolean {
        return ContextCompat.checkSelfPermission(this, permission) ==
                PackageManager.PERMISSION_GRANTED
    }

    private fun hasBluetoothConnectPermission(): Boolean {
        return Build.VERSION.SDK_INT < Build.VERSION_CODES.S ||
                hasPermission(Manifest.permission.BLUETOOTH_CONNECT)
    }

    private fun hasBluetoothScanPermission(): Boolean {
        return Build.VERSION.SDK_INT < Build.VERSION_CODES.S ||
                hasPermission(Manifest.permission.BLUETOOTH_SCAN)
    }

    private fun hasLocationPermission(): Boolean {
        return hasPermission(Manifest.permission.ACCESS_FINE_LOCATION)
    }

    private val bluetoothReceiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            when (intent?.action) {
                BluetoothDevice.ACTION_FOUND -> handleDeviceFound(intent)

                BluetoothAdapter.ACTION_DISCOVERY_FINISHED -> {
                    Log.d(TAG, "검색 완료")
                    btnScan.isEnabled = true
                    Toast.makeText(this@MainActivity, "블루투스 검색 완료",
Toast.LENGTH_SHORT).show()
                }
            }
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val manager = getSystemService(Context.BLUETOOTH_SERVICE) as
```

```
BluetoothManager
        bluetoothAdapter = manager.adapter

        btnScan = findViewById(R.id.btn_scan)
        btnReset = findViewById(R.id.btn_reset)
        btnModeToggle = findViewById(R.id.btn_mode_toggle)
        lvDevices = findViewById(R.id.lv_devices)

        btnfront = findViewById(R.id.btn_front)
        btnback = findViewById(R.id.btn_back)
        btnleft = findViewById(R.id.btn_left)
        btnright = findViewById(R.id.btn_right)
        btnstop = findViewById(R.id.btn_stop)

        listAdapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, ArrayList())
        lvDevices.adapter = listAdapter

        updateModeUI()

        btnScan.setOnClickListener { checkPermissionsAndScan() }

        btnReset.setOnClickListener {
            sendData("r")
            Toast.makeText(this, "리셋 전송", Toast.LENGTH_SHORT).show()
            Log.d(TAG, "리셋 전송: r")
        }

        lvDevices.setOnItemClickListener { _, _, position, _ ->
            val device = devicesList[position]
            connectToDevice(device)
        }

        btnModeToggle.setOnClickListener { toggleMode() }

        btnfront.setOnClickListener {
            if (!isTrackingMode) sendData("a")
        }
        btnback.setOnClickListener {
            if (!isTrackingMode) sendData("b")
        }
        btnleft.setOnClickListener {
            if (!isTrackingMode) sendData("c")
```

```
        }
        btnright.setOnClickListener {
            if (!isTrackingMode) sendData("d")
        }
        btnstop.setOnClickListener {
            if (!isTrackingMode) sendData("e")
        }

        registerReceiver(bluetoothReceiver, IntentFilter().apply {
            addAction(BluetoothDevice.ACTION_FOUND)
            addAction(BluetoothAdapter.ACTION_DISCOVERY_FINISHED)
        })
    }

    private fun handleDeviceFound(intent: Intent) {
        val device: BluetoothDevice? = if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
            intent.getParcelableExtra(BluetoothDevice.EXTRA_DEVICE,
BluetoothDevice::class.java)
        } else {
            @Suppress("DEPRECATION")
            intent.getParcelableExtra(BluetoothDevice.EXTRA_DEVICE)
        }

        device?.let {
            if (devicesList.none { d -> d.address == it.address }) {
                devicesList.add(it)

                val name = try {
                    if (hasBluetoothConnectPermission()) it.name ?: "Unknown"
                    else "권한 없음"
                } catch (e: SecurityException) {
                    "오류"
                }

                val address = try {
                    if (hasBluetoothConnectPermission()) it.address
                    else "비공개"
                } catch (e: SecurityException) {
                    "오류"
                }
```

```
                listAdapter.add("$name\n$address")
                listAdapter.notifyDataSetChanged()
            }
        }
    }

    private val requestPermissionsLauncher =
        registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) {
permissions ->
            Log.d(TAG, "권한 결과: $permissions")
            if (permissions.all { it.value }) {
                startBluetoothDiscovery()
            } else {
                Toast.makeText(this, "블루투스 권한이 필요합니다.",
Toast.LENGTH_SHORT).show()
            }
        }

    private fun checkPermissionsAndScan() {
        val list = mutableListOf<String>()

        if (!hasBluetoothScanPermission())
            list.add(Manifest.permission.BLUETOOTH_SCAN)

        if (!hasBluetoothConnectPermission())
            list.add(Manifest.permission.BLUETOOTH_CONNECT)

        if (!hasLocationPermission())
            list.add(Manifest.permission.ACCESS_FINE_LOCATION)

        if (list.isNotEmpty()) {
            requestPermissionsLauncher.launch(list.toTypedArray())
        } else {
            startBluetoothDiscovery()
        }
    }

    private fun startBluetoothDiscovery() {
        val adapter = bluetoothAdapter

        if (adapter == null) {
            Toast.makeText(this, "블루투스를 지원하지 않는 기기입니다.",
```

```
Toast.LENGTH_SHORT).show()
            return
        }

        if (!adapter.isEnabled) {
            Toast.makeText(this, "블루투스를 먼저 켜주세요.",
Toast.LENGTH_SHORT).show()
            return
        }

        if (hasBluetoothScanPermission()) {
            try {
                if (adapter.isDiscovering) {
                    adapter.cancelDiscovery()
                }
            } catch (e: SecurityException) {
                Log.e(TAG, "Discovery 권한 오류", e)
            }
        }

        devicesList.clear()
        listAdapter.clear()
        listAdapter.notifyDataSetChanged()

        val started = try {
            adapter.startDiscovery()
        } catch (e: SecurityException) {
            Log.e(TAG, "scan 권한 오류", e)
            false
        }

        Log.d(TAG, "스캔 시작: $started")

        if (started) {
            btnScan.isEnabled = false
            Toast.makeText(this, "블루투스 검색 시작", Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "블루투스 검색 시작 실패",
Toast.LENGTH_SHORT).show()
        }
    }
```

```
    private fun connectToDevice(device: BluetoothDevice) {
        if (!hasBluetoothConnectPermission()) {
            Toast.makeText(this, "BLUETOOTH_CONNECT 권한이 없습니다.",
Toast.LENGTH_SHORT).show()
            return
        }

        try {
            val name = device.name ?: "Unknown"
            Toast.makeText(this, "$name 연결 시도 중...", Toast.LENGTH_SHORT).show()
            bluetoothGatt = device.connectGatt(this, false, gattCallback)
        } catch (e: SecurityException) {
            Log.e(TAG, "connectGatt 오류", e)
            Toast.makeText(this, "연결 실패", Toast.LENGTH_SHORT).show()
        }
    }

    private val gattCallback = object : BluetoothGattCallback() {

        override fun onConnectionStateChange(
            gatt: BluetoothGatt?,
            status: Int,
            newState: Int
        ) {
            super.onConnectionStateChange(gatt, status, newState)

            if (newState == BluetoothProfile.STATE_CONNECTED) {
                Log.d(TAG, "연결 성공")

                runOnUiThread {
                    Toast.makeText(this@MainActivity, "블루투스 연결 성공",
Toast.LENGTH_SHORT).show()
                }

                try {
                    gatt?.discoverServices()
                } catch (e: SecurityException) {
                    Log.e(TAG, "discoverServices 오류", e)
                }
            } else if (newState == BluetoothProfile.STATE_DISCONNECTED) {
                Log.d(TAG, "연결 끊김")
```

```
                runOnUiThread {
                    Toast.makeText(this@MainActivity, "블루투스 연결 끊김",
Toast.LENGTH_SHORT).show()
                }

                writeCharacteristic = null
                bluetoothGatt = null
            }
        }

        override fun onServicesDiscovered(gatt: BluetoothGatt?, status: Int) {
            super.onServicesDiscovered(gatt, status)

            if (status == BluetoothGatt.GATT_SUCCESS) {
                val service = gatt?.getService(SERVICE_UUID)
                writeCharacteristic = service?.getCharacteristic(CHARACTERISTIC_UUID)

                if (writeCharacteristic != null) {
                    Log.d(TAG, "특성 설정 완료")
                    runOnUiThread {
                        Toast.makeText(this@MainActivity, "BLE 준비 완료",
Toast.LENGTH_SHORT).show()
                    }
                } else {
                    Log.e(TAG, "특성을 찾지 못했습니다.")
                    runOnUiThread {
                        Toast.makeText(this@MainActivity, "특성을 찾지 못했습니다.",
Toast.LENGTH_SHORT).show()
                    }
                }
            } else {
                Log.e(TAG, "서비스 탐색 실패: $status")
                runOnUiThread {
                    Toast.makeText(this@MainActivity, "서비스 탐색 실패",
Toast.LENGTH_SHORT).show()
                }
            }
        }

        override fun onCharacteristicWrite(
            gatt: BluetoothGatt?,
            characteristic: BluetoothGattCharacteristic?,
```

```
            status: Int
        ) {
            super.onCharacteristicWrite(gatt, characteristic, status)

            if (status == BluetoothGatt.GATT_SUCCESS) {
                Log.d(TAG, "쓰기 성공")
            } else {
                Log.e(TAG, "쓰기 실패: $status")
            }
        }
    }

    private fun sendData(data: String) {
        val gatt = bluetoothGatt
        val characteristic = writeCharacteristic

        if (gatt == null || characteristic == null) {
            Toast.makeText(this, "BLE가 연결되지 않았습니다.",
Toast.LENGTH_SHORT).show()
            return
        }

        if (!hasBluetoothConnectPermission()) {
            Toast.makeText(this, "BLUETOOTH_CONNECT 권한이 없습니다.",
Toast.LENGTH_SHORT).show()
            return
        }

        try {
            @Suppress("DEPRECATION")
            characteristic.value = data.toByteArray()

            characteristic.writeType = BluetoothGattCharacteristic.WRITE_TYPE_DEFAULT

            @Suppress("DEPRECATION")
            val success = gatt.writeCharacteristic(characteristic)
            Log.d(TAG, "전송: $data, success=$success")
        } catch (e: SecurityException) {
            Log.e(TAG, "전송 오류", e)
        }
    }
```

```
private fun setManualButtonsEnabled(enabled: Boolean) {
    btnfront.isEnabled = enabled
    btnback.isEnabled = enabled
    btnleft.isEnabled = enabled
    btnright.isEnabled = enabled
    btnstop.isEnabled = enabled

    val alpha = if (enabled) 1.0f else 0.3f
    btnfront.alpha = alpha
    btnback.alpha = alpha
    btnleft.alpha = alpha
    btnright.alpha = alpha
    btnstop.alpha = alpha
}

private fun updateModeUI() {
    if (isTrackingMode) {
        btnModeToggle.text = "모드: 추종"
        setManualButtonsEnabled(false)
    } else {
        btnModeToggle.text = "모드: 수동"
        setManualButtonsEnabled(true)
    }
}

private fun toggleMode() {
    isTrackingMode = !isTrackingMode
    updateModeUI()

    if (isTrackingMode) {
        sendData("t")
        Toast.makeText(this, "객체추종 모드", Toast.LENGTH_SHORT).show()
        Log.d(TAG, "추종 모드")
    } else {
        sendData("m")
        Toast.makeText(this, "수동 모드", Toast.LENGTH_SHORT).show()
        Log.d(TAG, "수동 모드")
    }
}

override fun onDestroy() {
    super.onDestroy()
```

```
        try {
            unregisterReceiver(bluetoothReceiver)
        } catch (e: IllegalArgumentException) {
            Log.w(TAG, "Receiver already unregistered")
        }

        if (hasBluetoothConnectPermission()) {
            try {
                bluetoothGatt?.close()
            } catch (e: SecurityException) {
                Log.e(TAG, "Gatt close 권한 오류", e)
            }
        }

        bluetoothGatt = null
        writeCharacteristic = null
    }
}
```

3) ATmega128 프로그램 소스 코드

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <stdint.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define DIR_STOP      0
#define DIR_FORWARD 1
#define DIR_REVERSE 2

#define MODE_MANUAL    0
#define MODE_TRACKING 1

// ==============================
// 초음파 핀 설정
// ==============================
#define FRONT_TRIG PORTA0
#define FRONT_ECHO PORTA1

#define LEFT_TRIG   PORTA2
#define LEFT_ECHO   PORTA3
```

```
#define RIGHT_TRIG PORTA4
#define RIGHT_ECHO PORTA5

// ==============================
// 장애물 기준 거리(cm)
// ==============================
#define OBS_FRONT_CM    5
#define OBS_SIDE_CM     5

// ==============================
// 회피용 PWM (16bit 기준)
// ==============================
#define AVOID_TURN_PWM 15000
#define AVOID_BACK_PWM 13000

// ==============================
// Timer2 for ultrasonic
// 16MHz / 64 = 250kHz
// 1 tick = 4us
// ==============================
volatile uint32_t g_t2_ovf = 0;

ISR(TIMER2_OVF_vect)
{
    g_t2_ovf++;
}

uint32_t timer2_now_ticks(void)
{
    uint32_t ovf1, ovf2;
    uint8_t tcnt;

    do {
        ovf1 = g_t2_ovf;
        tcnt = TCNT2;
        ovf2 = g_t2_ovf;
    } while (ovf1 != ovf2);

    return (ovf1 << 8) | tcnt;
}
```

```
void timer2_init_for_ultrasonic(void)
{
    TCCR2 = 0;
    TCNT2 = 0;
    TIMSK |= (1 << TOIE2);
    TCCR2 = (1 << CS22);    // prescaler 64
}

uint32_t ticks_to_us(uint32_t ticks)
{
    return ticks * 4UL;
}

uint16_t pulse_us_to_cm(uint32_t pulse_us)
{
    return (uint16_t)(pulse_us / 58UL);
}

// ==============================
// USART0 : Jetson Nano
// RXD0 = PORTE0
// TXD0 = PORTE1
// ==============================
void UART0_init(void) {
    UBRR0H = 0;
    UBRR0L = 103;    // 9600
    UCSR0A = 0;
    UCSR0B = (1 << RXEN0) | (1 << TXEN0);    // 수신 + 송신
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}

uint8_t UART0_available(void) {
    return (UCSR0A & (1 << RXC0));
}

uint8_t UART0_read(void) {
    return UDR0;
}

void UART0_sendByte(uint8_t data) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = data;
```

```
}

// ==============================
// USART1 : Bluetooth
// RXD1 = PORTD2
// TXD1 = PORTD3
// ==============================
void UART1_init(void) {
    UBRR1H = 0;
    UBRR1L = 103;    // 9600
    UCSR1A = 0;
    UCSR1B = (1 << RXEN1);
    UCSR1C = (1 << UCSZ11) | (1 << UCSZ10);
}

uint8_t UART1_available(void) {
    return (UCSR1A & (1 << RXC1));
}

uint8_t UART1_read(void) {
    return UDR1;
}

// ==============================
// PWM Timer1 (LEFT)
// ==============================
void PWM1_init(void) {
    DDRB |= (1 << PORTB5) | (1 << PORTB6);

    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);

    ICR1 = 2000;
    OCR1A = 0;
    OCR1B = 0;
}

// ==============================
// PWM Timer3 (RIGHT)
// ==============================
void PWM3_init(void) {
    DDRE |= (1 << PORTE3) | (1 << PORTE4);
```

```
    TCCR3A = (1 << COM3A1) | (1 << COM3B1) | (1 << WGM31);
    TCCR3B = (1 << WGM33) | (1 << WGM32) | (1 << CS31);

    ICR3 = 2000;
    OCR3A = 0;
    OCR3B = 0;
}

// ==============================
// 초음파 GPIO 초기화
// ==============================
void ultrasonic_init(void) {
    DDRA |= (1 << FRONT_TRIG) | (1 << LEFT_TRIG) | (1 << RIGHT_TRIG);
    DDRA &= ~((1 << FRONT_ECHO) | (1 << LEFT_ECHO) | (1 << RIGHT_ECHO));

    PORTA &= ~((1 << FRONT_TRIG) | (1 << LEFT_TRIG) | (1 << RIGHT_TRIG));
}

void trig_high(uint8_t trig_pin)
{
    PORTA |= (1 << trig_pin);
}

void trig_low(uint8_t trig_pin)
{
    PORTA &= ~(1 << trig_pin);
}

uint8_t echo_read(uint8_t echo_pin)
{
    return (PINA & (1 << echo_pin)) ? 1 : 0;
}

// ==============================
// Jetson 패킷 수신 (USART0)
// AA LDIR LL LH RDIR RL RH 55
// ==============================
uint8_t receive_packet_uart0_nonblocking(uint8_t *ldir, uint16_t *lpwm,
uint8_t *rdir, uint16_t *rpwm)
{
    static uint8_t buf[8];
```

```
    static uint8_t idx = 0;

    while (UART0_available()) {
        uint8_t b = UART0_read();

        if (idx == 0) {
            if (b == 0xAA) {
                buf[idx++] = b;
            }
            } else {
            buf[idx++] = b;

            if (idx == 8) {
                idx = 0;

                if (buf[7] == 0x55) {
                    *ldir = buf[1];
                    *lpwm = (uint16_t)buf[2] | ((uint16_t)buf[3] << 8);
                    *rdir = buf[4];
                    *rpwm = (uint16_t)buf[5] | ((uint16_t)buf[6] << 8);
                    return 1;
                    } else {
                    if (b == 0xAA) {
                        buf[0] = 0xAA;
                        idx = 1;
                    }
                }
            }
        }
    }

    return 0;
}

// ==============================
// 방향 전환 보호
// ==============================
uint8_t need_brake(uint8_t prev, uint8_t now)
{
    return ((prev == DIR_FORWARD && now == DIR_REVERSE) ||
    (prev == DIR_REVERSE && now == DIR_FORWARD));
}
```

```
// ==============================
// PWM scaling
// ==============================
uint16_t scale_pwm16_to_timer(uint16_t pwm16, uint16_t top_value)
{
    uint16_t scaled = ((uint32_t)pwm16 * top_value) / 65535UL;
    if (scaled < 100) scaled = 0;
    return scaled;
}

// ==============================
// 모터 적용
// ==============================
void apply_motor_command(uint8_t left_dir, uint16_t left_pwm,
uint8_t right_dir, uint16_t right_pwm,
uint8_t *prev_left_dir, uint8_t *prev_right_dir)
{
    uint16_t scaled_left  = scale_pwm16_to_timer(left_pwm, ICR1);
    uint16_t scaled_right = scale_pwm16_to_timer(right_pwm, ICR3);

    if (need_brake(*prev_left_dir, left_dir)) {
        OCR1A = 0;
        OCR1B = 0;
        _delay_ms(50);
    }

    if (need_brake(*prev_right_dir, right_dir)) {
        OCR3A = 0;
        OCR3B = 0;
        _delay_ms(50);
    }

    // LEFT
    if (left_dir == DIR_FORWARD) {
        OCR1A = scaled_left;
        OCR1B = 0;
    }
    else if (left_dir == DIR_REVERSE) {
        OCR1A = 0;
        OCR1B = scaled_left;
    }
```

```
    else {
        OCR1A = 0;
        OCR1B = 0;
    }

    // RIGHT
    if (right_dir == DIR_FORWARD) {
        OCR3A = scaled_right;
        OCR3B = 0;
    }
    else if (right_dir == DIR_REVERSE) {
        OCR3A = 0;
        OCR3B = scaled_right;
    }
    else {
        OCR3A = 0;
        OCR3B = 0;
    }

    *prev_left_dir = left_dir;
    *prev_right_dir = right_dir;
}

// ==============================
// 수동 모드 문자 -> 모터 명령
// a: 전진
// b: 후진
// c: 좌
// d: 우
// e: 정지
// ==============================
void manual_command_to_motor(uint8_t cmd,
uint8_t *left_dir, uint16_t *left_pwm,
uint8_t *right_dir, uint16_t *right_pwm)
{
    switch (cmd) {
        case 'a':    // forward
        *left_dir = DIR_FORWARD;
        *left_pwm = 22000;
        *right_dir = DIR_FORWARD;
        *right_pwm = 22000;
        break;
```

```
        case 'b':    // reverse
        *left_dir = DIR_REVERSE;
        *left_pwm = 22000;
        *right_dir = DIR_REVERSE;
        *right_pwm = 22000;
        break;

        case 'c':    // left
        *left_dir = DIR_STOP;
        *left_pwm = 0;
        *right_dir = DIR_FORWARD;
        *right_pwm = 22000;
        break;

        case 'd':    // right
        *left_dir = DIR_FORWARD;
        *left_pwm = 22000;
        *right_dir = DIR_STOP;
        *right_pwm = 0;
        break;

        case 'e':    // stop
        default:
        *left_dir = DIR_STOP;
        *left_pwm = 0;
        *right_dir = DIR_STOP;
        *right_pwm = 0;
        break;
    }
}

// ============================
// 초음파 non-blocking 상태머신
// ============================
typedef enum {
    US_IDLE = 0,
    US_TRIG_ON,
    US_WAIT_RISE,
    US_WAIT_FALL
} us_state_t;
```

```
typedef struct {
    uint8_t trig_pin;
    uint8_t echo_pin;
    uint16_t distance_cm;
} ultrasonic_sensor_t;

ultrasonic_sensor_t sensors[3] = {
    {FRONT_TRIG, FRONT_ECHO, 999},
    {LEFT_TRIG,  LEFT_ECHO,  999},
    {RIGHT_TRIG, RIGHT_ECHO, 999}
};

volatile us_state_t us_state = US_IDLE;
volatile uint8_t us_idx = 0;
volatile uint32_t us_t0 = 0;
volatile uint32_t us_rise_tick = 0;
volatile uint32_t us_cycle_tick = 0;

#define US_TRIG_PULSE_TICKS    3
#define US_WAIT_TIMEOUT_TICKS 7500
#define US_CYCLE_GAP_TICKS     5000

void ultrasonic_service(void)
{
    uint32_t now = timer2_now_ticks();
    ultrasonic_sensor_t *s = &sensors[us_idx];

    switch (us_state) {
        case US_IDLE:
        if ((now - us_cycle_tick) >= US_CYCLE_GAP_TICKS) {
            trig_high(s->trig_pin);
            us_t0 = now;
            us_state = US_TRIG_ON;
        }
        break;

        case US_TRIG_ON:
        if ((now - us_t0) >= US_TRIG_PULSE_TICKS) {
            trig_low(s->trig_pin);
            us_t0 = now;
            us_state = US_WAIT_RISE;
        }
```

```
            break;

            case US_WAIT_RISE:
            if (echo_read(s->echo_pin)) {
                us_rise_tick = now;
                us_state = US_WAIT_FALL;
            }
            else if ((now - us_t0) >= US_WAIT_TIMEOUT_TICKS) {
                s->distance_cm = 999;
                us_idx = (us_idx + 1) % 3;
                us_cycle_tick = now;
                us_state = US_IDLE;
            }
            break;

            case US_WAIT_FALL:
            if (!echo_read(s->echo_pin)) {
                uint32_t pulse_ticks = now - us_rise_tick;
                uint32_t pulse_us = ticks_to_us(pulse_ticks);
                s->distance_cm = pulse_us_to_cm(pulse_us);

                us_idx = (us_idx + 1) % 3;
                us_cycle_tick = now;
                us_state = US_IDLE;
            }
            else if ((now - us_rise_tick) >= US_WAIT_TIMEOUT_TICKS) {
                s->distance_cm = 999;
                us_idx = (us_idx + 1) % 3;
                us_cycle_tick = now;
                us_state = US_IDLE;
            }
            break;
        }
    }

    // ==============================
    // 장애물 회피
    // return 1 -> 회피 명령 사용
    // return 0 -> Jetson 명령 사용
    // ==============================
    void obstacle_avoidance(uint16_t front_cm, uint16_t left_cm, uint16_t right_cm,
    uint8_t *left_dir, uint16_t *left_pwm,
```

```
uint8_t *right_dir, uint16_t *right_pwm)
{
    if (front_cm <= OBS_FRONT_CM) {
        if (left_cm > right_cm && left_cm > OBS_SIDE_CM) {
            *left_dir = DIR_STOP;
            *left_pwm = 0;
            *right_dir = DIR_FORWARD;
            *right_pwm = AVOID_TURN_PWM;
        }
        else if (right_cm > left_cm && right_cm > OBS_SIDE_CM) {
            *left_dir = DIR_FORWARD;
            *left_pwm = AVOID_TURN_PWM;
            *right_dir = DIR_STOP;
            *right_pwm = 0;
        }
        else {
            *left_dir = DIR_REVERSE;
            *left_pwm = AVOID_BACK_PWM;
            *right_dir = DIR_REVERSE;
            *right_pwm = AVOID_BACK_PWM;
        }
        return;
    }

    if (left_cm <= OBS_SIDE_CM && right_cm > OBS_SIDE_CM) {
        *left_dir = DIR_FORWARD;
        *left_pwm = AVOID_TURN_PWM;
        *right_dir = DIR_STOP;
        *right_pwm = 0;
        return;
    }

    if (right_cm <= OBS_SIDE_CM && left_cm > OBS_SIDE_CM) {
        *left_dir = DIR_STOP;
        *left_pwm = 0;
        *right_dir = DIR_FORWARD;
        *right_pwm = AVOID_TURN_PWM;
        return;
    }

    if (left_cm <= OBS_SIDE_CM && right_cm <= OBS_SIDE_CM) {
        *left_dir = DIR_STOP;
```

```
        *left_pwm = 0;
        *right_dir = DIR_STOP;
        *right_pwm = 0;
        return;
    }

    return;
}

// ==============================
// 메인
// ==============================
int main(void)
{
    uint8_t mode_flag = MODE_MANUAL;

    // 블루투스 수동 명령 마지막 값
    uint8_t manual_cmd = 'e';

    // Jetson 마지막 명령
    uint8_t jetson_left_dir = DIR_STOP;
    uint8_t jetson_right_dir = DIR_STOP;
    uint16_t jetson_left_pwm = 0;
    uint16_t jetson_right_pwm = 0;

    // 실제 적용 명령
    uint8_t apply_left_dir, apply_right_dir;
    uint16_t apply_left_pwm, apply_right_pwm;

    uint8_t prev_left_dir = DIR_STOP;
    uint8_t prev_right_dir = DIR_STOP;

    uint16_t front_cm, left_cm, right_cm;

    UART0_init();                // Jetson
    UART1_init();                // Bluetooth
    PWM1_init();
    PWM3_init();
    ultrasonic_init();
    timer2_init_for_ultrasonic();
    sei();
```

```
while (1) {
    // ==========================
    // Bluetooth 문자 수신 (USART1)
    // ==========================
    if (UART1_available()) {
        uint8_t bt = UART1_read();

        if (bt == 'm') {
            mode_flag = MODE_MANUAL;
            manual_cmd = 'e';
        }
        else if (bt == 't') {
            mode_flag = MODE_TRACKING;
        }
        else if (bt == 'r') {
            UART0_sendByte('r');    // Jetson으로 리셋 신호 전달
        }
        else {
            // 수동 명령은 수동 모드에서만 저장
            if (mode_flag == MODE_MANUAL) {
                manual_cmd = bt;
            }
        }
    }

    // ==========================
    // 수동 모드
    // ==========================
    if (mode_flag == MODE_MANUAL) {
        manual_command_to_motor(manual_cmd,
        &apply_left_dir, &apply_left_pwm,
        &apply_right_dir, &apply_right_pwm);

        apply_motor_command(apply_left_dir, apply_left_pwm,
        apply_right_dir, apply_right_pwm,
        &prev_left_dir, &prev_right_dir);
    }

    // ==========================
    // 객체추종 모드
    // ==========================
    else {
```

```
            ultrasonic_service();

            front_cm = sensors[0].distance_cm;
            left_cm  = sensors[1].distance_cm;
            right_cm = sensors[2].distance_cm;

            receive_packet_uart0_nonblocking(&jetson_left_dir, &jetson_left_pwm,
            &jetson_right_dir, &jetson_right_pwm);

            apply_left_dir = jetson_left_dir;
            apply_left_pwm = jetson_left_pwm;
            apply_right_dir = jetson_right_dir;
            apply_right_pwm = jetson_right_pwm;

            obstacle_avoidance(front_cm, left_cm, right_cm,
            &apply_left_dir, &apply_left_pwm,
            &apply_right_dir, &apply_right_pwm);

            apply_motor_command(apply_left_dir, apply_left_pwm,
            apply_right_dir, apply_right_pwm,
            &prev_left_dir, &prev_right_dir);
        }
    }
}
```

4) Jetson nano 프로그램 소스 코드

```
import cv2
import torch
import time
import numpy as np
import serial
from collections import deque

from deep_sort_realtime.deepsort_tracker import DeepSort

from models.experimental import attempt_load
from utils.general import non_max_suppression, scale_coords
from utils.datasets import letterbox
from utils.torch_utils import select_device
```

```
INPUT_WIDTH = 1280
INPUT_HEIGHT = 720

DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 360

UART_PORT = "/dev/ttyS0"
UART_BAUD = 9600

DIR_STOP = 0
DIR_FORWARD = 1
DIR_REVERSE = 2

STOP_AREA_MIN = 90000
STOP_AREA_MAX = 140000


def gstreamer_pipeline():
    return (
        "nvarguscamerasrc ! "
        "video/x-raw(memory:NVMM), width=(int)1280, height=(int)720, "
        "format=(string)NV12, framerate=(fraction)30/1 ! "
        "nvvidconv ! "
        "video/x-raw, width=(int)640, height=(int)360, format=(string)BGRx ! "
        "videoconvert ! "
        "video/x-raw, format=(string)BGR ! "
        "appsink drop=true max-buffers=1 sync=false"
    )


def preprocess_image(img_bgr, img_size=224, stride=32):
    img = letterbox(img_bgr, new_shape=img_size, stride=stride)[0]
    img = img[:, :, ::-1].transpose(2, 0, 1)
    img = np.ascontiguousarray(img)
    return img


def draw_center_guides(frame, input_width=1280, dead_zone_ratio=0.1):
    h, w = frame.shape[:2]

    input_center_x = input_width / 2.0
    dead_zone = input_width * dead_zone_ratio
```

```
    left_input = input_center_x - dead_zone
    right_input = input_center_x + dead_zone

    scale_x = w / float(input_width)

    center_disp = int(input_center_x * scale_x)
    left_disp = int(left_input * scale_x)
    right_disp = int(right_input * scale_x)

    cv2.line(frame, (center_disp, 0), (center_disp, h), (0, 255, 255), 2)
    cv2.line(frame, (left_disp, 0), (left_disp, h), (255, 255, 0), 2)
    cv2.line(frame, (right_disp, 0), (right_disp, h), (255, 255, 0), 2)


def display_x_to_input_x(obj_center_x_display, display_width=640, input_width=1280):
    scale = input_width / float(display_width)
    return int(obj_center_x_display * scale)


def get_motor_command_by_area_and_x(
    obj_center_x_input,
    box_area,
    input_width=1280,
    stop_area_min=STOP_AREA_MIN,
    stop_area_max=STOP_AREA_MAX,
    base_speed=15000,
    max_correction=5000,
    dead_zone_ratio=0.1
):
    center_x = input_width / 2.0
    error_x = obj_center_x_input - center_x
    dead_zone = input_width * dead_zone_ratio

    if box_area < stop_area_min:
        move_mode = "FORWARD"
        base_dir = DIR_FORWARD
    elif box_area <= stop_area_max:
        return DIR_STOP, 0, DIR_STOP, 0, int(error_x), "STOP"
    else:
        move_mode = "REVERSE"
        base_dir = DIR_REVERSE
```

```
    if abs(error_x) <= dead_zone:
        return base_dir, base_speed, base_dir, base_speed, int(error_x), move_mode

    normalized_error = error_x / center_x
    normalized_error = max(-1.0, min(1.0, normalized_error))
    correction = int(abs(normalized_error) * max_correction)

    if move_mode == "FORWARD":
        if normalized_error < 0:
            left_dir = DIR_STOP
            right_dir = DIR_FORWARD
            left_pwm = 0
            right_pwm = base_speed + correction
            direction = "FORWARD-LEFT"
        else:
            left_dir = DIR_FORWARD
            right_dir = DIR_STOP
            left_pwm = base_speed + correction
            right_pwm = 0
            direction = "FORWARD-RIGHT"
    else:
        if normalized_error < 0:
            left_dir = DIR_STOP
            right_dir = DIR_REVERSE
            left_pwm = 0
            right_pwm = base_speed + correction
            direction = "REVERSE-LEFT"
        else:
            left_dir = DIR_REVERSE
            right_dir = DIR_STOP
            left_pwm = base_speed + correction
            right_pwm = 0
            direction = "REVERSE-RIGHT"

    left_pwm = max(0, min(65535, left_pwm))
    right_pwm = max(0, min(65535, right_pwm))

    return left_dir, left_pwm, right_dir, right_pwm, int(error_x), direction


def build_bbs_for_deepsort(det, names, target_class_name="person"):
```

```
    bbs = []

    for *xyxy, conf, cls in det:
        class_id = int(cls.item())
        class_name = names[class_id]

        if class_name != target_class_name:
            continue

        x1, y1, x2, y2 = [int(v.item()) for v in xyxy]
        w = x2 - x1
        h = y2 - y1

        if w <= 0 or h <= 0:
            continue

        bbs.append(([x1, y1, w, h], float(conf.item()), class_name))

    return bbs


def choose_target_track(tracks, frame_width, locked_id=None):
    center_x = frame_width // 2

    confirmed_tracks = [track for track in tracks if track.is_confirmed()]
    if not confirmed_tracks:
        return None, None

    if locked_id is not None:
        for track in confirmed_tracks:
            if track.track_id == locked_id:
                return track, locked_id

    best_track = None
    best_dist = float("inf")

    for track in confirmed_tracks:
        x1, y1, x2, y2 = map(int, track.to_ltrb())
        track_center_x = (x1 + x2) // 2
        dist = abs(track_center_x - center_x)

        if dist < best_dist:
```

```
                best_dist = dist
                best_track = track

        if best_track is None:
            return None, None

        return best_track, best_track.track_id


def send_uart_packet(ser, left_dir, left_pwm, right_dir, right_pwm):
        left_pwm = max(0, min(65535, int(left_pwm)))
        right_pwm = max(0, min(65535, int(right_pwm)))

        left_low = left_pwm & 0xFF
        left_high = (left_pwm >> 8) & 0xFF
        right_low = right_pwm & 0xFF
        right_high = (right_pwm >> 8) & 0xFF

        packet = bytes([
            0xAA,
            left_dir & 0xFF,
            left_low,
            left_high,
            right_dir & 0xFF,
            right_low,
            right_high,
            0x55
        ])
        ser.write(packet)

        return packet, left_low, left_high, right_low, right_high


def check_reset_command(ser):
        if ser is None:
            return False

        try:
            if ser.in_waiting > 0:
                rx = ser.read(ser.in_waiting)
                if b'r' in rx or b'R' in rx:
                    return True
```

```
    except Exception as e:
        print("UART 수신 오류:", repr(e))

    return False


def main():
    weights = "yolov5s.pt"
    img_size = 224
    conf_thres = 0.30
    iou_thres = 0.45
    target_class_name = "person"

    device = select_device('')
    print("Using device:", device)

    print("모델 로드 시작")
    model = attempt_load(weights, map_location=device)
    model.eval()
    stride = int(model.stride.max())
    names = model.module.names if hasattr(model, 'module') else model.names
    print("모델 로드 완료")

    print("DeepSORT 생성 시작")
    tracker = DeepSort(max_age=90)
    print("DeepSORT 생성 완료")

    print("UART 열기 시작")
    try:
        ser = serial.Serial(UART_PORT, UART_BAUD, timeout=0)
        time.sleep(2)
        print("UART 열기 성공:", UART_PORT, UART_BAUD)
    except Exception as e:
        print("UART 열기 실패:", repr(e))
        ser = None

    locked_target_id = None

    pipeline = gstreamer_pipeline()
    print("Using pipeline:")
    print(pipeline)
```

```
    cap = cv2.VideoCapture(pipeline, cv2.CAP_GSTREAMER)

    if not cap.isOpened():
        print("카메라를 열지 못했습니다.")
        if ser is not None:
            ser.close()
        return

    print("카메라 열기 성공")
    time.sleep(1.0)

    warmup_ok = False
    for _ in range(5):
        ret, frame = cap.read()
        if ret:
            warmup_ok = True
            break
        time.sleep(0.2)

    if not warmup_ok:
        print("카메라 워밍업 실패")
        cap.release()
        if ser is not None:
            ser.close()
        return

    cv2.namedWindow("YOLOv5 + DeepSORT Jetson Follow Test",
cv2.WINDOW_NORMAL)

    infer_fps_history = deque(maxlen=10)
    last_infer_time = None

    prev_packet_key = None

    frame_count = 0
    cached_target_track = None

    while True:
        ret, frame = cap.read()
        if not ret:
            print("프레임을 읽지 못했습니다.")
            break
```

```
        if check_reset_command(ser):
            locked_target_id = None
            cached_target_track = None
            prev_packet_key = None
            print("리셋 신호 수신: target lock 초기화")

        original = frame.copy()
        h0, w0 = original.shape[:2]

        draw_center_guides(frame, input_width=INPUT_WIDTH, dead_zone_ratio=0.1)

        direction_text = "NO TARGET"
        command_text = "STOP"
        left_dir = DIR_STOP
        right_dir = DIR_STOP
        left_pwm = 0
        right_pwm = 0
        error_x = 0
        obj_center_x_input = None
        uart_text = "UART: NONE"
        box_area = 0

        if frame_count % 2 == 0:
            infer_now = time.time()
            if last_infer_time is not None:
                infer_inst_fps = 1.0 / (infer_now - last_infer_time) if infer_now !=
last_infer_time else 0.0
                infer_fps_history.append(infer_inst_fps)
            last_infer_time = infer_now

            img = preprocess_image(original, img_size=img_size, stride=stride)
            img = torch.from_numpy(img).to(device)
            img = img.float() / 255.0

            if img.ndimension() == 3:
                img = img.unsqueeze(0)

            with torch.no_grad():
                pred = model(img, augment=False)[0]

            pred = non_max_suppression(pred, conf_thres, iou_thres)
```

```
            all_bbs = []
            for det in pred:
                if det is not None and len(det):
                    det[:, :4] = scale_coords(img.shape[2:], det[:, :4],
original.shape).round()
                    bbs = build_bbs_for_deepsort(det, names,
target_class_name=target_class_name)
                    all_bbs.extend(bbs)

            tracks = tracker.update_tracks(all_bbs, frame=original)
            cached_target_track, locked_target_id = choose_target_track(
                tracks, w0, locked_id=locked_target_id
            )

        target_track = cached_target_track

        if target_track is not None and target_track.is_confirmed():
            x1, y1, x2, y2 = map(int, target_track.to_ltrb())
            obj_center_x_display = (x1 + x2) // 2
            obj_center_y = (y1 + y2) // 2
            box_area = (x2 - x1) * (y2 - y1)

            obj_center_x_input = display_x_to_input_x(
                obj_center_x_display,
                display_width=w0,
                input_width=INPUT_WIDTH
            )

            left_dir, left_pwm, right_dir, right_pwm, error_x, direction_text =
get_motor_command_by_area_and_x(
                obj_center_x_input,
                box_area=box_area,
                input_width=INPUT_WIDTH,
                stop_area_min=STOP_AREA_MIN,
                stop_area_max=STOP_AREA_MAX,
                base_speed=22000,
                max_correction=12000,
                dead_zone_ratio=0.1
            )

            command_text = f"LD={left_dir},LP={left_pwm},
```

```
RD={right_dir},RP={right_pwm}"

                if ser is not None:
                    packet_key = (left_dir, left_pwm, right_dir, right_pwm)
                    if packet_key != prev_packet_key:
                        packet, ll, lh, rl, rh = send_uart_packet(
                            ser, left_dir, left_pwm, right_dir, right_pwm
                        )
                        uart_text = f"UART: SENT AA {left_dir:02X} {ll:02X} {lh:02X}
{right_dir:02X} {rl:02X} {rh:02X} 55"
                        prev_packet_key = packet_key
                    else:
                        uart_text = "UART: SAME, NOT SENT"
                else:
                    uart_text = "UART: SERIAL FAIL"

                cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 0, 0), 3)
                cv2.circle(frame, (obj_center_x_display, obj_center_y), 5, (0, 0, 255), -1)

                label = f"TARGET ID {target_track.track_id}"
                cv2.putText(frame, label, (x1, max(y1 - 35, 0)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)

                cv2.line(frame, (obj_center_x_display, 0), (obj_center_x_display, h0), (0, 0, 255),
2)

            else:
                locked_target_id = None
                cached_target_track = None

                if ser is not None:
                    packet_key = (DIR_STOP, 0, DIR_STOP, 0)
                    if packet_key != prev_packet_key:
                        packet, ll, lh, rl, rh = send_uart_packet(
                            ser, DIR_STOP, 0, DIR_STOP, 0
                        )
                        uart_text = f"UART: SENT AA 00 {ll:02X} {lh:02X} 00 {rl:02X}
{rh:02X} 55"
                        prev_packet_key = packet_key
                    else:
                        uart_text = "UART: SAME, NOT SENT"
                else:
```

```
                uart_text = "UART: SERIAL FAIL"

        infer_fps = sum(infer_fps_history) / len(infer_fps_history) if len(infer_fps_history)
> 0 else 0.0

        cv2.putText(frame, f"Direction: {direction_text}", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
        cv2.putText(frame, command_text, (10, 65),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.60, (255, 255, 0), 2)
        cv2.putText(frame, f"Locked ID: {locked_target_id}", (10, 100),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 200, 0), 2)
        cv2.putText(frame, f"Error X(input): {error_x}", (10, 135),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (180, 255, 180), 2)
        cv2.putText(frame, f"Box Area: {box_area}", (10, 170),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (200, 255, 200), 2)
        cv2.putText(frame, f"Area Min: {STOP_AREA_MIN}  Max: {STOP_AREA_MAX}",
(10, 205),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.70, (255, 255, 255), 2)

        if obj_center_x_input is not None:
            cv2.putText(frame, f"Target X(input): {obj_center_x_input}", (10, 240),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (180, 220, 255), 2)

        cv2.putText(frame, uart_text, (10, 275),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 180, 180), 2)

        cv2.putText(frame, f"Infer FPS(avg10): {infer_fps:.2f}", (10, 310),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 200, 255), 2)

        cv2.imshow("YOLOv5 + DeepSORT Jetson Follow Test", frame)

        key = cv2.waitKey(1) & 0xFF
        if key == 27 or key == ord('q'):
            break

        frame_count += 1

    cap.release()
    cv2.destroyAllWindows()

    if ser is not None:
        ser.close()
```

```
if __name__ == "__main__":
    main()
```